

HỌC MÁY CƠ BẢN VỚI PYTHON

Chương 1 – Buổi 2

Chương trình học máy đầu tiên
(Your First Machine Learning Program)

Lê Nhật Tùng

Bài giảng đi kèm giáo trình

Mục tiêu buổi học

Sau buổi học này, bạn có thể:

- Tự viết, từng bước, chương trình học máy đầu tiên bằng Python với bộ dữ liệu **Iris**.
- Chia dữ liệu thành **tập huấn luyện** (*training set*) và **tập kiểm tra** (*test set*); hiểu `stratify`.
- Tính tay dự đoán của **k láng giềng gần nhất** (*k-nearest neighbors, kNN*).
- Tính và giải thích **độ chính xác** (*accuracy*); tìm mẫu bị dự đoán sai.
- Nhận biết **quá khớp** (*overfitting*) và **chưa khớp** (*underfitting*) bằng cách so sánh kết quả huấn luyện và kiểm tra.

Nội dung

1. Chương trình học máy đầu tiên

2. Khái quát hóa: học hiểu hay học vẹt?

3. Ứng dụng và giới hạn

4. Thực hành có hướng dẫn

5. Tổng kết

Phần 1

Chương trình học máy đầu tiên

Bài toán và công cụ

- **Bài toán**: cho 4 số đo của một bông hoa, dự đoán bông hoa thuộc loài nào.
  $\Rightarrow$ **phân loại** (*classification*) với **3 lớp**.
- **Ngôn ngữ**: Python.
- **Thư viện**: `scikit-learn` (đọc "sai-kít-lơn") – thư viện học máy phổ biến nhất của Python, có sẵn bộ dữ liệu Iris và các thuật toán.
- **Môi trường**: **Google Colab** – chỉ cần tài khoản Google, không cần cài đặt.

Mẹo nhỏ

1. Vào `https://colab.research.google.com`.
2. Mở notebook `Chuong01_HocMayCoBan.ipynb`.
3. Chạy từng ô bằng **Shift + Enter**.

Lưu ý

Hôm nay chưa cần hiểu hết từng dòng lệnh – mục tiêu là thấy **bức tranh toàn cảnh**. Chương 2 giải thích kỹ các thư viện.

Bước 1: Nạp thư viện

ch01.py — ô import_thu_vien

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_iris
4 from sklearn.model_selection import train_test_split
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.metrics import accuracy_score
7
8 pd.set_option("display.max_columns", None)  # hiện đủ mọi cột khi in bảng
9 pd.set_option("display.width", 85)          # in gọn trong khổ 85 ký tự
```

- `pandas`: làm việc với **bảng dữ liệu** (*DataFrame*). `matplotlib`: vẽ biểu đồ.
- `load_iris`: nạp bộ dữ liệu Iris. `train_test_split`: chia tập huấn luyện / kiểm tra.
- `KNeighborsClassifier`: thuật toán **k láng giềng gần nhất** (*k-Nearest Neighbors, kNN*).
- `accuracy_score`: tính **độ chính xác** (*accuracy*). Dòng 8–9: chỉ để bảng được in đầy đủ, gọn gàng.

Bước 2: Nạp và xem dữ liệu

ch01.py — ô iris_load

```python
iris = load_iris(as_frame=True)     # tải bộ dữ liệu Iris có sẵn trong scikit-learn
df = iris.frame                     # bảng dữ liệu dạng pandas DataFrame
print("Kích thước bảng:", df.shape) # (số dòng, số cột)
print(df.head())                    # xem 5 dòng đầu
```

Kết quả

```
Kích thước bảng: (150, 5)
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target
0                5.1               3.5                1.4               0.2       0
1                4.9               3.0                1.4               0.2       0
2                4.7               3.2                1.3               0.2       0
3                4.6               3.1                1.5               0.2       0
4                5.0               3.6                1.4               0.2       0
```

- `df.shape` = `(150, 5)`: 150 dòng (150 bông hoa) và 5 cột – 4 cột đặc trưng và cột nhãn `target`.
- `df.head()` in 5 dòng đầu. Nhãn được **mã hóa thành số** (*label encoding*): 0 = setosa, 1 = versicolor, 2 = virginica.

Tên đặc trưng, tên nhãn và số mẫu mỗi lớp

ch01.py — ô iris_ten

```
print("Tên các đặc trưng:", iris.feature_names)
print("Tên các nhãn     :", iris.target_names.tolist())
print("Số mẫu mỗi loài:")
print(df["target"].value_counts().sort_index())
```

Kết quả

```
Tên các đặc trưng: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Tên các nhãn     : ['setosa', 'versicolor', 'virginica']
Số mẫu mỗi loài:
target
0    50
1    50
2    50
Name: count, dtype: int64
```

- Mỗi loài có đúng 50 mẫu: số mẫu giữa các lớp bằng nhau ⇒ dữ liệu **cân bằng** (*balanced*).
- Nếu một lớp rất ít mẫu (ví dụ 1% giao dịch gian lận), ta gọi là **mất cân bằng** (*imbalanced*) – sẽ gặp ở Chương 8.

Kích thước trung bình của từng loài

ch01.py — ô iris_tb_loai

```
# Kích thước trung bình của mỗi loài (cm), làm tròn 2 chữ số
print(df.groupby("target").mean().round(2))
```

Kết quả

target	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.01	3.43	1.46	0.25
1	5.94	2.77	4.26	1.33
2	6.59	2.97	5.55	2.03

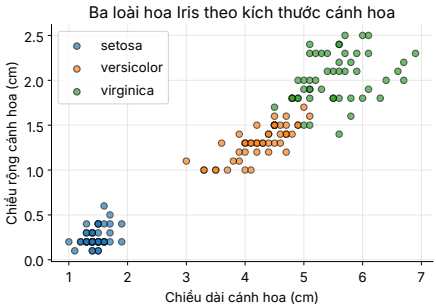
- `groupby("target")` gom các dòng cùng loài; `mean()` tính trung bình từng cột trong mỗi nhóm.
- Cánh hoa setosa (loài 0) dài 1,46 cm – chỉ bằng **một phần ba** hai loài còn lại.
- Versicolor (1) và virginica (2) khác nhau ít hơn: 4,26 so với 5,55 cm.
- Chiều rộng đài hoa của ba loài gần như bằng nhau $\Rightarrow$ ít giúp phân biệt.

Bước 3: Khám phá dữ liệu bằng biểu đồ

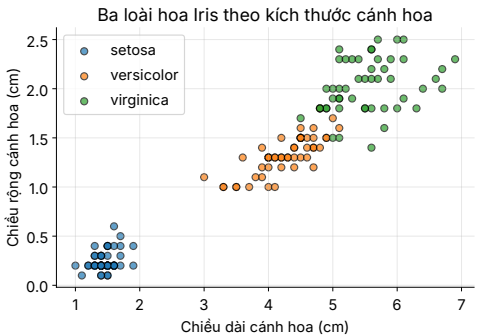
Biểu đồ phân tán (*scatter plot*): mỗi bông hoa là một chấm; trục ngang = chiều dài cánh hoa, trục dọc = chiều rộng cánh hoa, màu = loài.

ch01.py — ô iris_scatter

```
ten_loai = iris.target_names
mau = ["tab:blue", "tab:orange", "tab:green"]
for nhan in range(3):
    phan = df[df["target"] == nhan]
    plt.scatter(phan["petal length (cm)"], phan["petal width (cm)"],
                c=mau[nhan], label=ten_loai[nhan], alpha=0.7, edgecolors="k")
plt.xlabel("Chiều dài cánh hoa (cm)")
plt.ylabel("Chiều rộng cánh hoa (cm)")
plt.title("Ba loài hoa Iris theo kích thước cánh hoa")
plt.legend()
plt.show()
```



Đọc biểu đồ: dữ liệu “nói” gì?

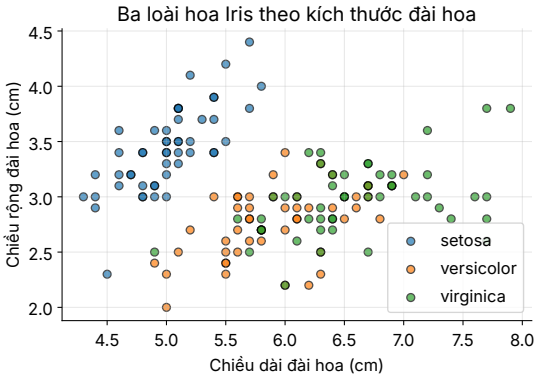


- *Setosa* (xanh dương) có cánh hoa nhỏ và **tách biệt hẳn** hai loài còn lại.
- *Versicolor* (cam) và *virginica* (xanh lá) có cánh hoa lớn hơn, nằm **sát nhau** và **chồng lấn một chút**.
- Dự đoán trước: mô hình sẽ nhận ra *setosa* rất dễ; lỗi (nếu có) sẽ nằm ở **ranh giới** giữa *versicolor* và *virginica*.

Mẹo nhỏ

Luôn **nhìn dữ liệu** trước khi huấn luyện: biểu đồ giúp đặt kỳ vọng và phát hiện vấn đề sớm.

So sánh: vẽ theo kích thước đài hoa

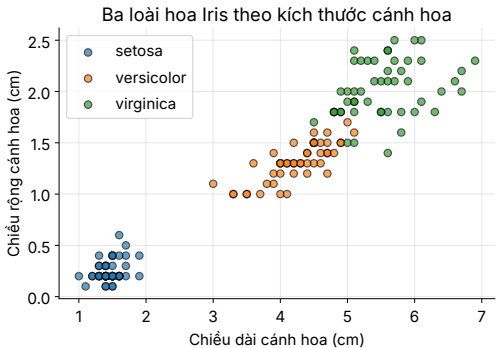


- Mã giống hệt, chỉ đổi tên hai cột.
- Setosa vẫn tách riêng (đài ngắn và rộng).
- Versicolor và virginica **trộn lẫn** vào nhau: chỉ nhìn đài hoa, cả con người cũng khó phân biệt.

Lưu ý

Chất lượng của đặc trưng quyết định độ khó của bài toán. Khám phá dữ liệu giúp biết trước đặc trưng nào có ích.

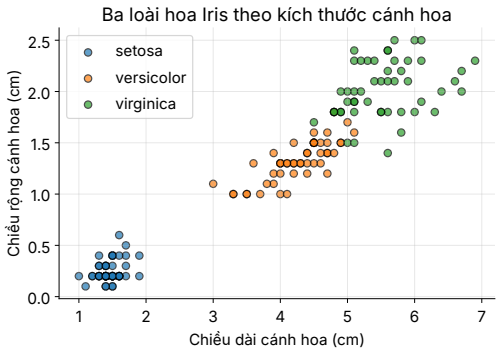
Bài tập mẫu: đoán loài bằng mắt



Bài tập mẫu: Kích thước cánh hoa (dài $\times$ rộng, cm)

(a) $1{,}6 \times 0{,}4$; (b) $4{,}2 \times 1{,}3$; (c) $6{,}0 \times 2{,}2$; (d) $4{,}9 \times 1{,}6$?

Bài tập mẫu: đoán loài bằng mắt



Bài tập mẫu: Kích thước cánh hoa (dài $\times$ rộng, cm)

(a) $1{,}6 \times 0{,}4$; (b) $4{,}2 \times 1{,}3$; (c) $6{,}0 \times 2{,}2$; (d) $4{,}9 \times 1{,}6$?

Lời giải.

- (a) góc dưới trái, đám xanh dương $\Rightarrow$ setosa.
- (b) giữa đám cam $\Rightarrow$ versicolor.
- (c) góc trên phải, đám xanh lá $\Rightarrow$ virginica.
- (d) rơi đúng **vùng chồng lấn** $\Rightarrow$ khó đoán nhất.

Bạn vừa làm đúng việc kNN sẽ làm: **nhìn những bông hoa ở gần để đoán**.

Bước 4a: Tách đặc trưng X và nhãn y

ch01.py — ô tach_X_y

```
X = df[iris.feature_names]   # ma trận đặc trưng: 150 dòng x 4 cột
y = df["target"]             # vector nhãn: 150 giá trị 0, 1, 2
print("X có kích thước:", X.shape)
print("y có kích thước:", y.shape)
```

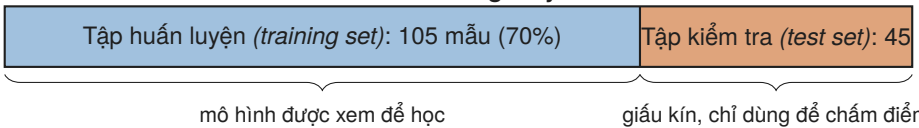
Kết quả

```
X có kích thước: (150, 4)
y có kích thước: (150,)
```

- `X`: **ma trận đặc trưng** (*feature matrix*) – 150 dòng $\times$ 4 cột, đúng ký hiệu X kích thước $n \times d$.
- `y`: **vector nhãn** (*target vector*) – 150 giá trị 0, 1, 2. Kích thước `(150,)` nghĩa là **một chiều**, 150 phần tử.

Bước 4b: Vì sao phải chia tập huấn luyện và tập kiểm tra?

Để biết mô hình có thực sự "hiểu" hay chỉ "học thuộc lòng", phải kiểm tra nó trên những bông hoa **nó chưa từng thấy**.



Ví dụ: Ôn thi bằng ngân hàng đề

Sinh viên ôn đúng 100 câu trong ngân hàng đề. Nếu đề thi lấy y nguyên 100 câu đó, học thuộc lòng cũng được 10 điểm – ta không biết em có thật sự hiểu bài. Chỉ khi ra **câu hỏi mới** mới đánh giá đúng năng lực. Tập kiểm tra chính là "đề thi có câu hỏi mới" dành cho mô hình.

Bước 4c: Chia dữ liệu bằng `train_test_split`

ch01.py — ô iris_split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
print("Tập huấn luyện:", X_train.shape, " Tập kiểm tra:", X_test.shape)
```

Kết quả

```
Tập huấn luyện: (105, 4)  Tập kiểm tra: (45, 4)
```

Hàm `train_test_split` **trộn ngẫu nhiên** (*shuffle*) dữ liệu rồi chia. Ý nghĩa các tham số:

- `test_size=0.3`: dành 30% cho tập kiểm tra (45 mẫu), 70% còn lại để huấn luyện (105 mẫu).
- `random_state=42`: cố định cách trộn để lần chạy nào cũng cho cùng kết quả (*reproducibility*). Số 42 không có gì đặc biệt.
- `stratify=y`: **chia phân tầng** (*stratified split*) – tỉ lệ ba loài trong mỗi tập giống tỉ lệ ban đầu (mỗi loài khoảng 1/3).
- Quy ước tên biến: `X_train`, `y_train` (huấn luyện); `X_test`, `y_test` (kiểm tra).

Tác dụng của `stratify`: kiểm tra bằng mã

ch01.py — ô kiem_tra_stratify

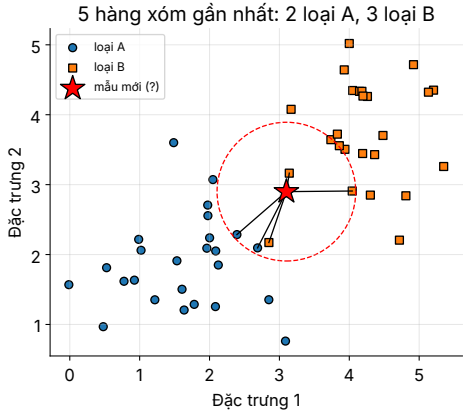
```
print("Số mẫu mỗi loài trong tập huấn luyện:", y_train.value_counts().sort_index().tolist())
print("Số mẫu mỗi loài trong tập kiểm tra  :", y_test.value_counts().sort_index().tolist())
# Thử chia KHÔNG có stratify để so sánh
_, _, _, y_test_ko = train_test_split(X, y, test_size=0.3, random_state=7)
print("Không stratify (random_state=7), tập kiểm tra:",
      y_test_ko.value_counts().sort_index().tolist())
```

Kết quả

```
Số mẫu mỗi loài trong tập huấn luyện: [35, 35, 35]
Số mẫu mỗi loài trong tập kiểm tra  : [15, 15, 15]
Không stratify (random_state=7), tập kiểm tra: [12, 16, 17]
```

- Có `stratify`: mỗi loài đúng **35** bông huấn luyện và **15** bông kiểm tra.
- Không có `stratify`: tập kiểm tra có 12 setosa nhưng 17 virginica – **lệch**.
- Dữ liệu nhỏ mà lệch ⇒ kết quả đánh giá kém tin cậy. Với phân loại, luôn dùng `stratify=y`.

Bước 5a: Thuật toán k láng giềng gần nhất (kNN)



$k = 5$: trong vòng tròn có 2 mẫu loại A, 3 mẫu loại B $\Rightarrow$ dự đoán **loại B**.

k láng giềng gần nhất (*k-Nearest Neighbors, kNN*): *"Hãy cho tôi biết bạn chơi với ai, tôi sẽ nói bạn là người thế nào."*

1. Tính **khoảng cách** (*distance*) từ mẫu mới tới mọi mẫu huấn luyện.
2. Lấy k mẫu **gần nhất** – các "hàng xóm" (*neighbors*).
3. Chọn nhãn **chiếm đa số** trong k hàng xóm (*majority vote*).

Chương 3: cách đo "khoảng cách". Chương 9: học kNN thật kỹ.

Ví dụ đời thường: đoán khu phố giàu hay nghèo

Ví dụ đời thường: "Nhìn hàng xóm để đoán"

Bạn bị dẫn bịt mắt đến một con phố lạ rồi mở mắt ra. Muốn đoán khu nhà giàu hay bình dân, bạn nhìn những căn nhà **xung quanh**: nếu 4 trong 5 căn gần nhất là biệt thự $\Rightarrow$ gần như chắc chắn là khu nhà giàu.

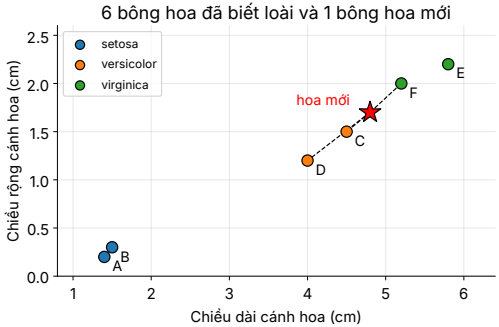
Toán dễ hiểu: Đo "gần" bằng khoảng cách (định lý Pytago)

Với hai đặc trưng:

$$\text{khoảng cách} = \sqrt{(\text{chênh lệch chiều dài})^2 + (\text{chênh lệch chiều rộng})^2}$$

Chương 3 mở rộng cho nhiều đặc trưng: **khoảng cách Euclid** (*Euclidean distance*).

Bài tập mẫu: kNN tính tay với 6 bông hoa



- A $(1{,}4;\ 0{,}2)$, B $(1{,}5;\ 0{,}3)$: setosa.
- C $(4{,}5;\ 1{,}5)$, D $(4{,}0;\ 1{,}2)$: versicolor.
- E $(5{,}8;\ 2{,}2)$, F $(5{,}2;\ 2{,}0)$: virginica.
- Hoa mới: cánh hoa $(4{,}8;\ 1{,}7)$.

Câu hỏi cho lớp

Dùng kNN với $k = 3$: bông hoa mới thuộc loài nào? Hãy tính trước khi xem lời giải.

Lời giải: ba bước của kNN

Bước 1 – khoảng cách (ví dụ A: $\sqrt{3{,}4^2 + 1{,}5^2} = \sqrt{13{,}81} \approx 3{,}716$):

Hoa	Loài	Chênh dài	Chênh rộng	Tổng bình phương	Khoảng cách
A	setosa	3,4	1,5	$11{,}56 + 2{,}25 = 13{,}81$	3,716
B	setosa	3,3	1,4	$10{,}89 + 1{,}96 = 12{,}85$	3,585
C	versicolor	0,3	0,2	$0{,}09 + 0{,}04 = 0{,}13$	**0,361**
D	versicolor	0,8	0,5	$0{,}64 + 0{,}25 = 0{,}89$	**0,943**
E	virginica	1,0	0,5	$1{,}00 + 0{,}25 = 1{,}25$	1,118
F	virginica	0,4	0,3	$0{,}16 + 0{,}09 = 0{,}25$	**0,500**

Bước 2 – sắp xếp: C (0,361), F (0,500), D (0,943), E, B, A $\Rightarrow$ 3 bông gần nhất: **C, F, D**.

Bước 3 – bỏ phiếu (*majority vote*): versicolor 2 phiếu (C, D), virginica 1 phiếu (F) $\Rightarrow$ **versicolor**.

Kiểm tra lời giải bằng `scikit-learn`

ch01.py — ô knn_tay

```python
# Kiểm tra lại Bài tập mẫu tính tay: 6 bông hoa, chỉ dùng 2 số đo cánh hoa
X6 = [[1.4, 0.2], [1.5, 0.3],      # setosa
      [4.5, 1.5], [4.0, 1.2],      # versicolor
      [5.8, 2.2], [5.2, 2.0]]      # virginica
y6 = ["setosa", "setosa", "versicolor", "versicolor", "virginica", "virginica"]
hoa_moi = [[4.8, 1.7]]
knn3 = KNeighborsClassifier(n_neighbors=3).fit(X6, y6)
khoang_cach, chi_so = knn3.kneighbors(hoa_moi)   # 3 hàng xóm gần nhất
for kc, i in zip(khoang_cach[0], chi_so[0]):
    print(f"Hàng xóm {X6[i]} - {y6[i]:10s} cách {kc:.3f}")
print("Dự đoán với k = 3:", knn3.predict(hoa_moi)[0])
```

Kết quả

```
Hàng xóm [4.5, 1.5] - versicolor cách 0.361
Hàng xóm [5.2, 2.0] - virginica  cách 0.500
Hàng xóm [4.0, 1.2] - versicolor cách 0.943
Dự đoán với k = 3: versicolor
```

Hàm `kneighbors` trả về khoảng cách và vị trí các hàng xóm gần nhất: **khớp hoàn toàn** với bảng tính tay. Hai setosa A, B ở rất xa nên không ảnh hưởng (trừ khi k rất lớn).

Bước 5b: Huấn luyện mô hình chỉ với hai dòng lệnh

ch01.py — ô iris_train

```
mo_hinh = KNeighborsClassifier(n_neighbors=5)   # "hỏi ý kiến 5 hàng xóm gần nhất"
mo_hinh.fit(X_train, y_train)                   # bước HỌC: ghi nhớ dữ liệu huấn luyện
print("Đã huấn luyện xong mô hình:", mo_hinh)
```

Kết quả

```
Đã huấn luyện xong mô hình: KNeighborsClassifier()
```

- Dòng 1: **tạo** mô hình kNN với $k = 5$ (`n_neighbors=5`).
- Dòng 2: `.fit(X_train, y_train)` – bước **huấn luyện** (*fitting*). Với kNN, "học" đơn giản là **ghi nhớ** dữ liệu huấn luyện; việc tính toán diễn ra lúc dự đoán.

Mẹo nhỏ

Mọi mô hình trong `scikit-learn` đều dùng chung cách gọi: **tạo mô hình** → `.fit(X_train, y_train)`. Bạn sẽ gặp lại mẫu câu lệnh này trong mọi chương.

Bước 6a: Dự đoán trên tập kiểm tra

ch01.py — ô iris_predict

```
y_du_doan = mo_hinh.predict(X_test)            # bước DỰ ĐOÁN trên dữ liệu chưa thấy
so_sanh = pd.DataFrame({"Thực tế": y_test.values[:8],
                        "Dự đoán": y_du_doan[:8]})
print(so_sanh)
```

- `.predict(X_test)`: mô hình dự đoán loài cho 45 bông hoa **chưa từng thấy**.
- Ta đặt 8 dự đoán đầu cạnh **giá trị thật** (*ground truth*) để so sánh.
- Cả 8 dự đoán đều khớp. Để có con số tổng quát cho cả 45 bông, ta tính **độ chính xác** (*accuracy*).

Kết quả

	Thực tế	Dự đoán
0	2	2
1	1	1
2	2	2
3	1	1
4	2	2
5	2	2
6	1	1
7	1	1

Bước 6b: Đánh giá bằng độ chính xác

Toán dễ hiểu: Độ chính xác – Accuracy

Tỉ lệ số mẫu được dự đoán đúng trên tổng số mẫu:

$$\text{Accuracy} = \frac{\text{số mẫu dự đoán đúng}}{\text{tổng số mẫu}} \in [0, 1], \qquad \text{ví dụ: } \frac{44}{45} \approx 0{,}9778 = 97{,}78\%.$$

0 là sai hết, 1 là đúng hết; thường đổi ra phần trăm.

ch01.py — ô iris_accuracy

```
do_chinh_xac = accuracy_score(y_test, y_du_doan)
so_dung = (y_test.values == y_du_doan).sum()
print(f"Đoán đúng {so_dung}/{len(y_test)} bông hoa")
print(f"Độ chính xác (accuracy) = {do_chinh_xac:.4f}")
```

Kết quả

```
Đoán đúng 44/45 bông hoa
Độ chính xác (accuracy) = 0.9778
```

Chỉ với vài dòng lệnh, mô hình đoán đúng **44 trên 45** bông hoa nó chưa từng thấy – đúng như dự đoán từ biểu đồ phân tán.

Bài tập mẫu: tính accuracy bằng tay

Dự đoán cho 10 bông hoa (S = setosa, Ve = versicolor, Vi = virginica):

Bông hoa	1	2	3	4	5	6	7	8	9	10
Thực tế	S	Ve	Vi	Vi	S	Ve	Ve	Vi	S	Vi
Dự đoán	S	Ve	Vi	Ve	S	Ve	Vi	Vi	S	Vi

Bài tập mẫu: tính accuracy bằng tay

Dự đoán cho 10 bông hoa (S = setosa, Ve = versicolor, Vi = virginica):

Bông hoa	1	2	3	4	5	6	7	8	9	10
Thực tế	S	Ve	Vi	Vi	S	Ve	Ve	Vi	S	Vi
Dự đoán	S	Ve	Vi	Ve	S	Ve	Vi	Vi	S	Vi

Bài tập mẫu

Lời giải. Bông 4 và bông 7 sai, 8 bông còn lại đúng $\Rightarrow$ Accuracy $= 8/10 = 0{,}8 = 80\%$.
Cả hai lỗi đều là nhầm **versicolor – virginica** – đúng như dự đoán từ biểu đồ phân tán.

Bông hoa nào bị đoán sai? – phân tích lỗi

ch01.py — ô iris_sai

```python
# Bông hoa nào bị đoán sai? Lọc những dòng mà thực tế khác dự đoán
sai = X_test[y_test.values != y_du_doan].copy()
sai["thực tế"] = iris.target_names[y_test.values[y_test.values != y_du_doan]]
sai["dự đoán"] = iris.target_names[y_du_doan[y_test.values != y_du_doan]]
print(sai)
```

Kết quả

```
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
106                4.9               2.5                4.5               1.7

       thực tế     dự đoán
106  virginica  versicolor
```

- Bông số 106: thật ra là virginica, bị đoán là versicolor.
- Cánh hoa $4{,}5 \times 1{,}7$ cm – ngắn như versicolor (TB $4{,}26 \times 1{,}33$) hơn là virginica (TB $5{,}55 \times 2{,}03$).
- Một bông hoa “lạc loài” trong vùng chồng lấn: chỉ nhìn số đo, chuyên gia cũng dễ nhầm.
- Thói quen xem lại các mẫu sai gọi là **phân tích lỗi** *(error analysis)* (Chương 17).

Bước 7: Dùng mô hình cho dữ liệu mới

Giả sử bạn vừa đo hai bông hoa ngoài vườn và đưa số đo vào mô hình:

ch01.py — ô iris_new

```
hoa_moi = pd.DataFrame([[5.0, 3.4, 1.5, 0.2],    # bông hoa nhỏ
                        [6.7, 3.0, 5.2, 2.3]],   # bông hoa lớn
                       columns=iris.feature_names)
du_doan = mo_hinh.predict(hoa_moi)
for i, nhan in enumerate(du_doan):
    print(f"Bông hoa {i + 1} được dự đoán là: {iris.target_names[nhan]}")
```

Kết quả

```
Bông hoa 1 được dự đoán là: setosa
Bông hoa 2 được dự đoán là: virginica
```

- Bông 1: cánh hoa nhỏ (1,5 cm $\times$ 0,2 cm) $\Rightarrow$ *setosa*.
- Bông 2: cánh hoa lớn $\Rightarrow$ *virginica*.

Lưu ý

Phải nhập số đo **đúng thứ tự 4 cột** như lúc huấn luyện. Đưa chiều rộng vào chỗ chiều dài $\Rightarrow$ dự đoán sai mà **không báo lỗi**.

Thử thay đổi số hàng xóm k: siêu tham số

ch01.py — ô iris_k

```
for k in [1, 3, 5, 15, 51]:
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
    print(f"k = {k:2d}  ->  accuracy = {m.score(X_test, y_test):.4f}")
```

Kết quả

```
k =  1  ->  accuracy = 0.9333
k =  3  ->  accuracy = 0.9556
k =  5  ->  accuracy = 0.9778
k = 15  ->  accuracy = 0.9556
k = 51  ->  accuracy = 0.8889
```

- $k = 1$: chỉ hỏi **một** hàng xóm $\Rightarrow$ dễ bị một bông hoa "lạ" đánh lừa.
- $k = 51$: hỏi cả những bông hoa ở **rất xa** $\Rightarrow$ kém hơn.

Định nghĩa: Siêu tham số – Hyperparameter

Con số do **con người chọn trước** khi huấn luyện, không do mô hình tự học. Với kNN: số hàng xóm k.

Phân biệt với **tham số** (*parameter*): giá trị mô hình **tự học** từ dữ liệu (ví dụ các hệ số của hồi quy tuyến tính, Chương 5).

Chọn siêu tham số tốt: Chương 16.

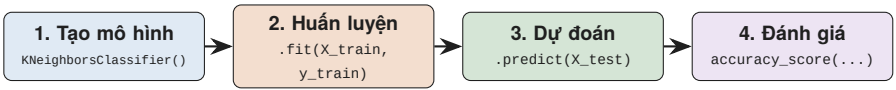
Ví dụ đời thường: hỏi ý kiến bao nhiêu người?

Ví dụ đời thường: Chọn k giống chọn số người để hỏi

Bạn phân vân có nên xem một bộ phim.

- Hỏi **1** người bạn: dễ gặp người có gu "lạ" $\Rightarrow k$ quá nhỏ.
- Hỏi **5** người bạn thân có gu giống mình: đáng tin hơn $\Rightarrow k$ vừa phải.
- Hỏi cả **100** người, kể cả người chẳng có gu giống bạn: ý kiến chung chẳng còn nói gì về sở thích của bạn $\Rightarrow k$ quá lớn.

Tổng kết: khuôn mẫu chung của scikit-learn



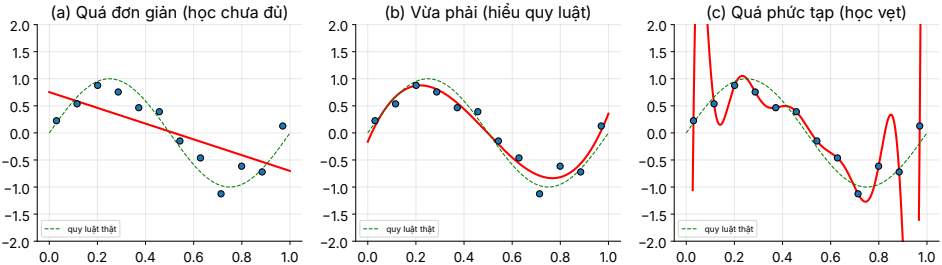
- Trong `scikit-learn`, mỗi thuật toán là một **bộ ước lượng** (*estimator*) có cùng các phương thức `fit`, `predict`, `score`.
- Đổi thuật toán (cây quyết định, SVM, mạng nơ-ron…) chỉ cần **đổi dòng tạo mô hình**; phần còn lại giữ nguyên.
- Đây là lý do ta dùng `scikit-learn` xuyên suốt giáo trình.

Phần 2

Khái quát hóa: học hiểu hay học vẹt?

Khái quát hóa

Mục tiêu cuối cùng không phải là làm tốt trên dữ liệu **đã thấy**, mà là làm tốt trên **dữ liệu mới**. Khả năng này gọi là **khái quát hóa** (*generalization*).



Mô hình (đường đỏ) học từ 12 điểm dữ liệu (chấm xanh); quy luật thật là đường đứt nét xanh lá.
(a) quá đơn giản (b) vừa phải, bám sát quy luật thật (c) quá phức tạp, uốn lượn vô lý giữa các điểm.

Chưa khớp và quá khớp

Định nghĩa: Chưa khớp – Underfitting

Mô hình **quá đơn giản**, làm kém **ngay trên dữ liệu huấn luyện**. Không nắm được quy luật.

Giống học sinh **chưa học bài**.

Định nghĩa: Quá khớp – Overfitting

Mô hình làm **rất tốt trên dữ liệu huấn luyện** nhưng **kém trên dữ liệu mới**, vì nó "học thuộc" cả những **nhiễu** (*noise*) ngẫu nhiên.

Giống học sinh **học vẹt**.

	Trên tập huấn luyện	**Trên tập kiểm tra**
Chưa khớp *(underfitting)*	kém	kém
Vừa khớp *(good fit)*	tốt	tốt
Quá khớp *(overfitting)*	rất tốt	kém

Đây là khái niệm **trung tâm** của học máy – ta sẽ gặp lại nhiều lần, đặc biệt ở Chương 6.

Ví dụ đời thường: ba kiểu học sinh

Ví dụ đời thường: Cùng ôn 100 câu trong ngân hàng đề

- (a) Đọc lướt: đề ôn 4 điểm, đi thi 4 điểm $\Rightarrow$ **chưa khớp**.
- (b) Hiểu cách giải từng dạng: đề ôn 9, đi thi 8,5 $\Rightarrow$ **khái quát hóa tốt**.
- (c) Học thuộc đáp án từng câu: đề ôn 10, gặp đề mới 5 điểm $\Rightarrow$ **quá khớp**.

Mẹo nhỏ

Dấu hiệu học vẹt: điểm **ôn tập** (tập huấn luyện) **cao hơn hẳn** điểm **thi thật** (tập kiểm tra). Ta kiểm tra điều này với kNN ngay sau đây.

Phát hiện học vẹt: accuracy huấn luyện so với kiểm tra

ch01.py — ô hoc_vet

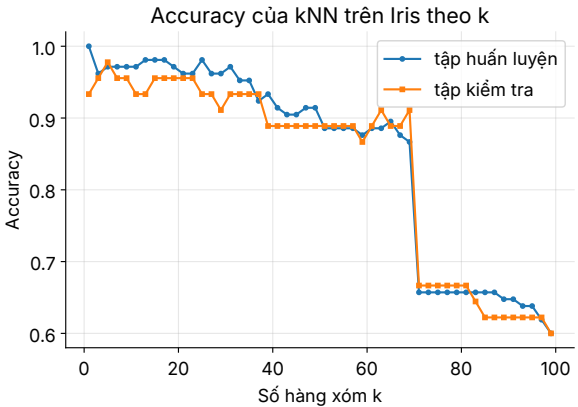
```
print("  k   accuracy huấn luyện   accuracy kiểm tra")
for k in [1, 5, 15, 51, 100]:
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
    print(f"{k:3d}         {m.score(X_train, y_train):.4f}
    ↪  {m.score(X_test, y_test):.4f}")
```

Kết quả

```
  k   accuracy huấn luyện   accuracy kiểm tra
  1         1.0000              0.9333
  5         0.9714              0.9778
 15         0.9810              0.9556
 51         0.8857              0.8889
100         0.5810              0.5778
```

- $k = 1$: huấn luyện **1,0** (hàng xóm gần nhất là *chính nó*), kiểm tra 0,93 $\Rightarrow$ **quá khớp**.
- $k = 5, 15$: hai con số gần nhau và cao $\Rightarrow$ tốt.
- $k = 100$: hỏi 100 trên 105 bông, dự đoán gần như luôn giống nhau $\Rightarrow$ **chưa khớp**.

Accuracy theo k: vùng quá khớp và chưa khớp



- **Trái** (k rất nhỏ): đường huấn luyện ở trên cao, tách khỏi đường kiểm tra $\Rightarrow$ quá khớp.
- **Phải** (k rất lớn): cả hai đường cùng rơi $\Rightarrow$ chưa khớp.
- **Giữa**: mô hình "vừa đủ".

Câu hỏi cho lớp

Mô hình có accuracy huấn luyện 0,99, kiểm tra 0,70: bị gì?

Phần 3

Ứng dụng và giới hạn

Một số ứng dụng học máy quen thuộc

Lĩnh vực	Ứng dụng	Loại bài toán
Thương mại điện tử	Gợi ý sản phẩm *(recommendation)*, dự đoán khách hàng rời bỏ *(churn)*	Phân loại, phân cụm
Ngân hàng	Chấm điểm tín dụng *(credit scoring)*, phát hiện gian lận *(fraud detection)*	Phân loại
Bất động sản	Định giá nhà đất	Hồi quy
Y tế	Hỗ trợ chẩn đoán từ ảnh X-quang, dự báo nguy cơ bệnh	Phân loại
Giao thông	Dự báo thời gian di chuyển, ùn tắc	Hồi quy
Nông nghiệp	Nhận diện sâu bệnh qua ảnh lá cây	Phân loại
Giáo dục	Cảnh báo sớm sinh viên có nguy cơ bỏ học	Phân loại

Câu hỏi cho lớp

Kể thêm ba ứng dụng học máy bạn gặp hằng ngày trên điện thoại. Mỗi ứng dụng là bài toán gì, nhãn là gì?

Giới hạn của học máy

- **Phụ thuộc dữ liệu**: dữ liệu sai lệch thì mô hình sai lệch – "rác vào, rác ra" (*garbage in, garbage out*).
- **Có thể thiên vị** (*bias*): nếu dữ liệu quá khứ chứa định kiến (ví dụ ít cho vay với một nhóm người), mô hình sẽ **học và lặp lại** định kiến đó.
- **Khó giải thích** (*interpretability*): nhiều mô hình mạnh hoạt động như "hộp đen" (*black box*) – đưa ra kết quả nhưng khó nói vì sao.
- **Không bao giờ đúng 100%**: mọi dự đoán đều có thể sai; với các quyết định quan trọng (y tế, tài chính, pháp lý), **con người vẫn phải kiểm tra lại**.

Phần 4

Thực hành có hướng dẫn

Thực hành 1.1: Phân loại rượu vang (Wine) – Bước 1

Bối cảnh. Bộ dữ liệu **Wine** (có sẵn trong `scikit-learn`): kết quả phân tích hóa học của **178** mẫu rượu vang trồng ở cùng một vùng nước Ý, thuộc **3 giống nho** (`class_0`, `class_1`, `class_2`). Mỗi mẫu có **13 đặc trưng**: độ cồn (*alcohol*), axit malic (*malic acid*), cường độ màu (*color intensity*)…

Yêu cầu. Lặp lại quy trình đã làm với Iris để xây dựng mô hình kNN dự đoán giống nho.

ch01.py — ô wine_buoc1

```python
from sklearn.datasets import load_wine
wine = load_wine(as_frame=True)
print(wine.frame.shape)
print(wine.target_names)
print(wine.frame["target"].value_counts().sort_index())
```

Kết quả

```
(178, 14)
['class_0' 'class_1' 'class_2']
target
0    59
1    71
2    48
Name: count, dtype: int64
```

178 dòng, 14 cột (13 đặc trưng + 1 nhãn). Số mẫu mỗi lớp không bằng nhau nhưng **không quá chênh lệch**.

Thực hành 1.1: Bước 2–3 – chia dữ liệu, huấn luyện, đánh giá

ch01.py — ô wine_buoc2

```
Xw = wine.data
yw = wine.target
Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    Xw, yw, test_size=0.3, random_state=42, stratify=yw)
print(Xw_train.shape, Xw_test.shape)
```

Kết quả

```
(124, 13) (54, 13)
```

ch01.py — ô wine_buoc3

```
knn_wine = KNeighborsClassifier(n_neighbors=5)
knn_wine.fit(Xw_train, yw_train)
print("Accuracy trên Wine:", round(knn_wine.score(Xw_test,
    yw_test), 4))
```

Kết quả

```
Accuracy trên Wine: 0.7222
```

Tách X, y; chia 70/30 có `stratify`; kNN với $k = 5$ – giống hệt Iris.

Câu hỏi cho lớp

Cùng thuật toán, cùng quy trình – vì sao Wine chỉ đạt khoảng **72%**, trong khi Iris đạt gần 98%?

Thực hành 1.1: Bước 4 – vì sao chỉ 72%?

ch01.py — ô wine_buoc4

```
# Thang đo của vài cột: giá trị nhỏ nhất và lớn nhất
print(Xw[["alcohol", "hue", "magnesium", "proline"]].agg(["min", "max"]).round(2))
```

Kết quả

	alcohol	hue	magnesium	proline
min	11.03	0.48	70.0	278.0
max	14.83	1.71	162.0	1680.0

Cột `proline` từ 278 đến 1 680, cột `hue` chỉ từ 0,48 đến 1,71. Khi kNN tính khoảng cách, chênh lệch `proline` (hàng trăm) lớn gấp hàng nghìn lần chênh lệch `hue` (dưới 1) $\Rightarrow$ khoảng cách gần như chỉ do `proline` quyết định.

Thực hành 1.1: Vì sao kết quả thấp?

- Các cột có **thang đo** (*scale*) **rất khác nhau**: `proline` từ vài trăm đến hơn 1 000, trong khi `hue` chỉ khoảng 0,5–1,7.
- kNN đo **khoảng cách** giữa các mẫu, nên cột có giá trị lớn **lấn át** hoàn toàn các cột còn lại: khoảng cách gần như chỉ phụ thuộc vào `proline`.
- Nguyên nhân **không** nằm ở thuật toán mà nằm ở **dữ liệu**.

Mẹo nhỏ

Chương 4 sẽ dùng kỹ thuật **chuẩn hóa** (*standardization / feature scaling*) để nâng độ chính xác lên khoảng **95%** mà **không cần đổi thuật toán** – minh chứng rõ ràng cho ý "dữ liệu tốt quan trọng hơn thuật toán".

Bảng thuật ngữ Việt – Anh của buổi học

Tiếng Việt	Tiếng Anh
Thư viện	Library
Khung dữ liệu	DataFrame
Biểu đồ phân tán	Scatter plot
Dữ liệu cân bằng	Balanced data
Tập huấn luyện	Training set
Tập kiểm tra	Test set
Chia phân tầng	Stratified split
k láng giềng gần nhất	k-nearest neighbors (kNN)

Tiếng Việt	Tiếng Anh
Khoảng cách Euclid	Euclidean distance
Bỏ phiếu đa số	Majority vote
Độ chính xác	Accuracy
Phân tích lỗi	Error analysis
Siêu tham số	Hyperparameter
Khái quát hóa	Generalization
Quá khớp	Overfitting
Chưa khớp	Underfitting

Phần 5

Tổng kết

Lỗi thường gặp

1. **Đánh giá mô hình trên chính tập huấn luyện** $\Rightarrow$ điểm số cao giả tạo. Luôn đánh giá trên tập kiểm tra mà mô hình chưa thấy.
2. **Quên** `random_state` $\Rightarrow$ mỗi lần chạy một kết quả khác nhau, khó so sánh và khó tìm lỗi.
3. **Nhập dữ liệu mới sai thứ tự cột** $\Rightarrow$ mô hình chỉ "biết" cột thứ nhất, thứ hai… chứ không hiểu ý nghĩa; dự đoán sai mà không báo lỗi.
4. **Dùng học máy cho bài toán đã có công thức**: nếu một câu lệnh `if` giải quyết được, đừng huấn luyện mô hình.
5. **Nhầm hồi quy với phân loại khi nhãn là số**: nhãn Iris là 0, 1, 2 nhưng đó là **mã** của loài hoa $\Rightarrow$ vẫn là phân loại.

Tóm tắt

- **Học máy** (*machine learning*) giúp máy tính **tự rút ra quy tắc từ dữ liệu** thay vì được lập trình tường minh.
- Dữ liệu gồm các **mẫu** (*samples*) (dòng), **đặc trưng** (*features*) (cột đầu vào, ma trận X kích thước $n \times d$) và **nhãn** (*labels*) (đáp án, vector y).
- **Học có giám sát** (*supervised*): **hồi quy** (*regression*) (nhãn là số), **phân loại** (*classification*) (nhãn là nhóm). **Học không giám sát** (*unsupervised*): tìm cấu trúc trong dữ liệu không nhãn, ví dụ **phân cụm** (*clustering*).
- Một dự án đi qua **7 bước**; công việc với dữ liệu chiếm phần lớn thời gian.
- `scikit-learn`: `model.fit(X_train, y_train)` rồi `model.predict(X_test)`.
- Luôn chia **tập huấn luyện** (*training set*) và **tập kiểm tra** (*test set*); mục tiêu là **khái quát hóa** (*generalization*) tốt, tránh **quá khớp** (*overfitting*) và **chưa khớp** (*underfitting*).

Câu hỏi thảo luận

1. Trong Bài tập mẫu kNN tính tay, nếu dùng $k = 5$ thì kết quả bỏ phiếu thế nào? Bạn thấy vấn đề gì?
2. Vì sao kNN làm kém trên Wine dù làm rất tốt trên Iris?
3. Vì sao không được đánh giá mô hình trên tập huấn luyện? Cho một ví dụ đời thường của riêng bạn.
4. Siêu tham số là gì? Cho biết siêu tham số của kNN. Ý nghĩa của `stratify=y`?
5. Trong hình "học hiểu hay học vẹt", mô hình nào dự đoán tốt nhất cho một điểm mới có $x = 0{,}5$?

Bài tập Buổi 1.2

- ⋆ **1.6**: Chạy lại chương trình Iris với `test_size=0.2`. Độ chính xác thay đổi thế nào?
- ⋆ **1.7**: Đổi `random_state` = 0, 1, 2, 3, 4. Vì sao kết quả khác nhau?
- ⋆ **1.8**: Dự đoán loài cho $(5{,}8;\ 2{,}7;\ 4{,}1;\ 1{,}0)$, $(7{,}2;\ 3{,}2;\ 6{,}0;\ 1{,}8)$, $(4{,}6;\ 3{,}2;\ 1{,}4;\ 0{,}2)$.
- ⋆ **1.9**: Tính tay kNN ($k = 3$) cho hoa $(2{,}0;\ 0{,}5)$ và $(5{,}5;\ 2{,}1)$ với 6 bông A–F.
- ⋆ **1.10**: Đúng 57/60 mẫu: accuracy? Sai thêm 3 mẫu thì giảm bao nhiêu điểm %?
- ⋆⋆ **1.11**: kNN chỉ với 2 cột đài hoa, rồi chỉ 2 cột cánh hoa; nhận xét về **chọn đặc trưng** (*feature selection*).
- ⋆⋆ **1.12**: Wine, $k = 1, 3, \ldots, 25$: vẽ accuracy huấn luyện và kiểm tra theo k.
- ⋆⋆ **1.13**: Khám phá `load_breast_cancer`; huấn luyện kNN.
- ⋆⋆⋆ **1.14**: Tự viết `du_doan_knn(X_train, y_train, x_moi, k)` không dùng `scikit-learn`.

Chi tiết: *Bài tập Buổi 1.2*, Chương 1. Buổi sau: **Chương 2 – Python cho học máy**.